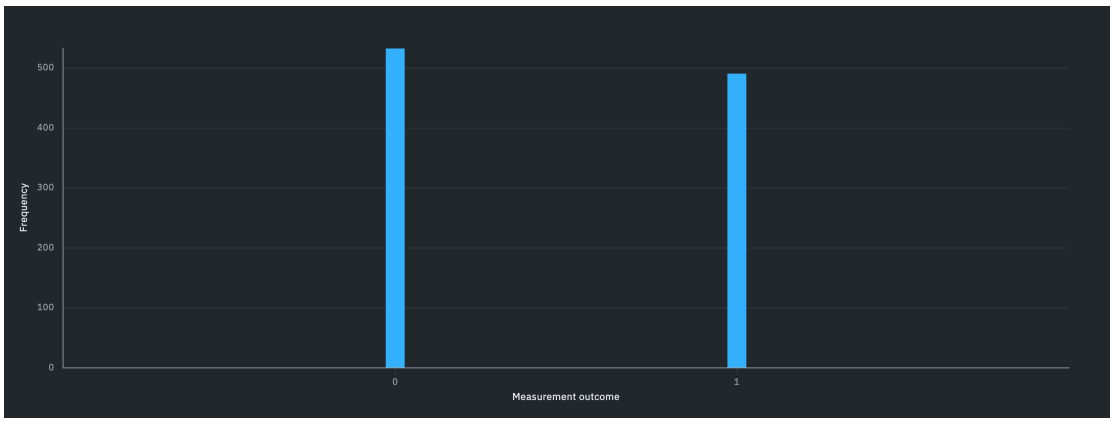
Lab 2 Meeting 2

PHYS:4340L Matthew Jarek

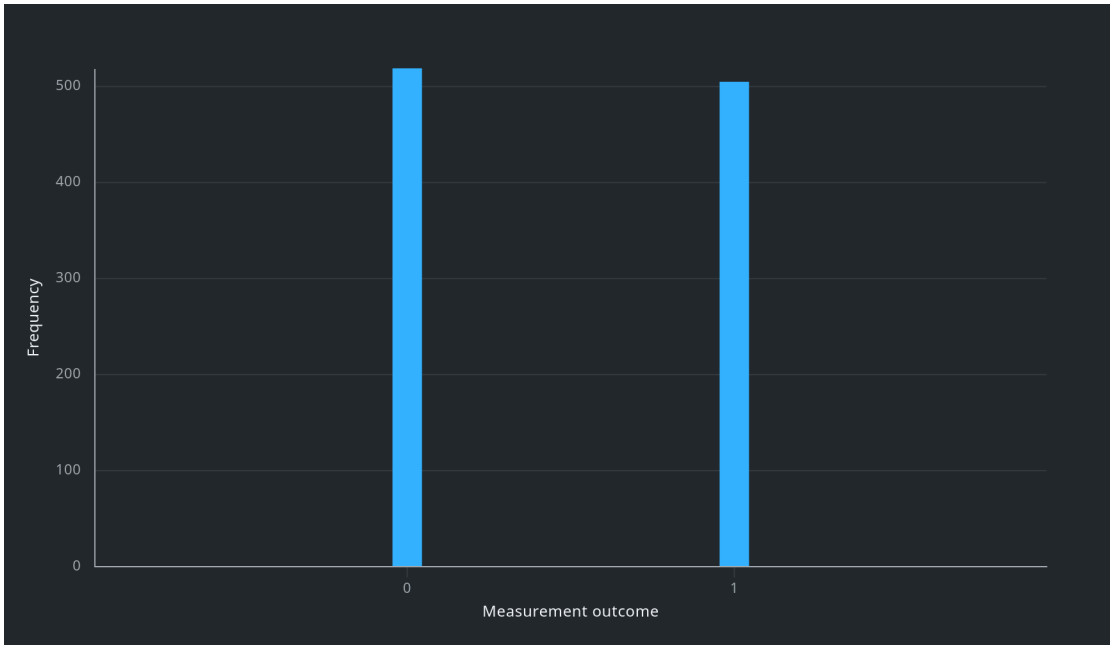
Circuit 1: Single-Qubit Superosition

Probability Histograms

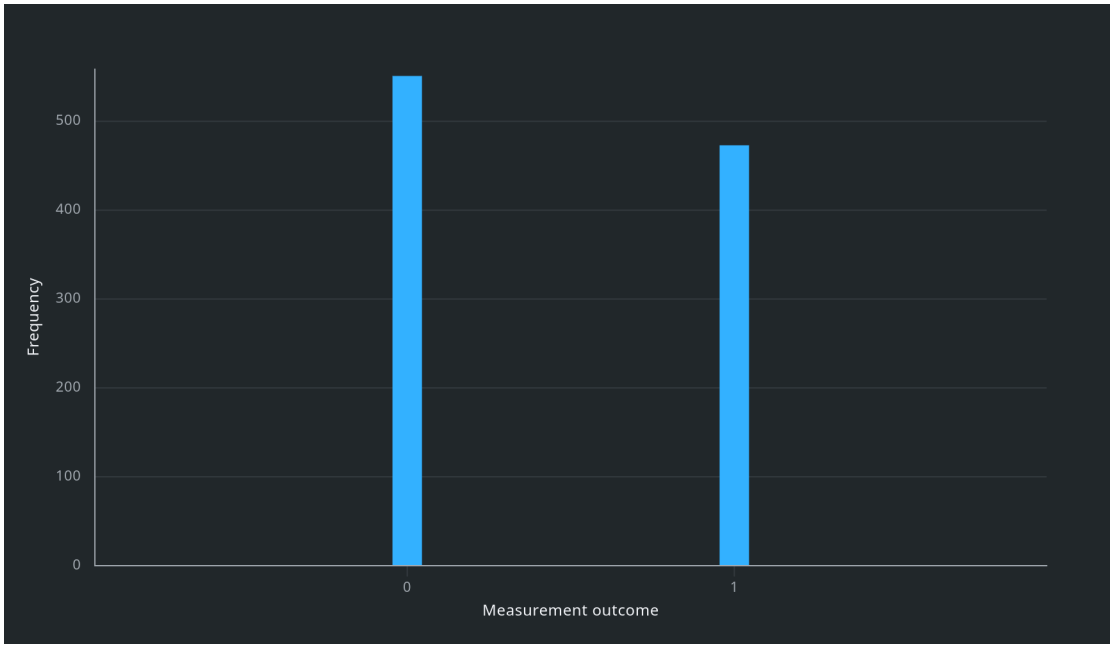
Run 1:



Run 2:



Run 3:

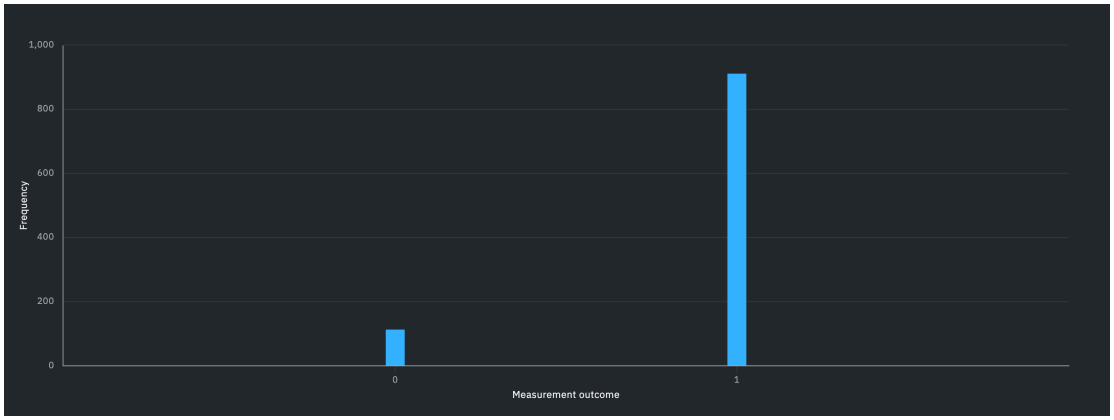


Run	|0⟩ Counts	|1⟩ Counts	Job ID
Run 1	533	491	d651b8rtraac73bj3cg0
Run 2	519	505	d651bortraac73bj3d70
Run 3	551	473	d651g6bc4tus73fgg7ig
Average	534.33	489.67	NA

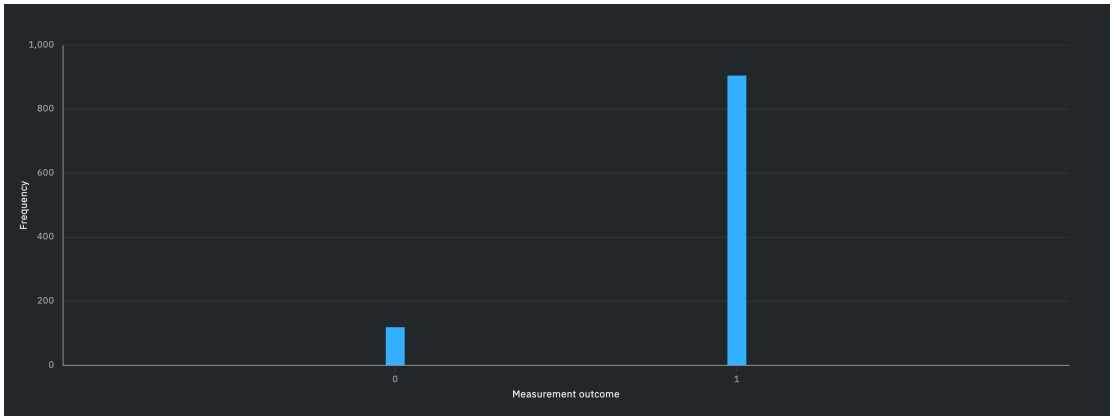
Circuit 2: Pauli X Flip

Probability Histograms

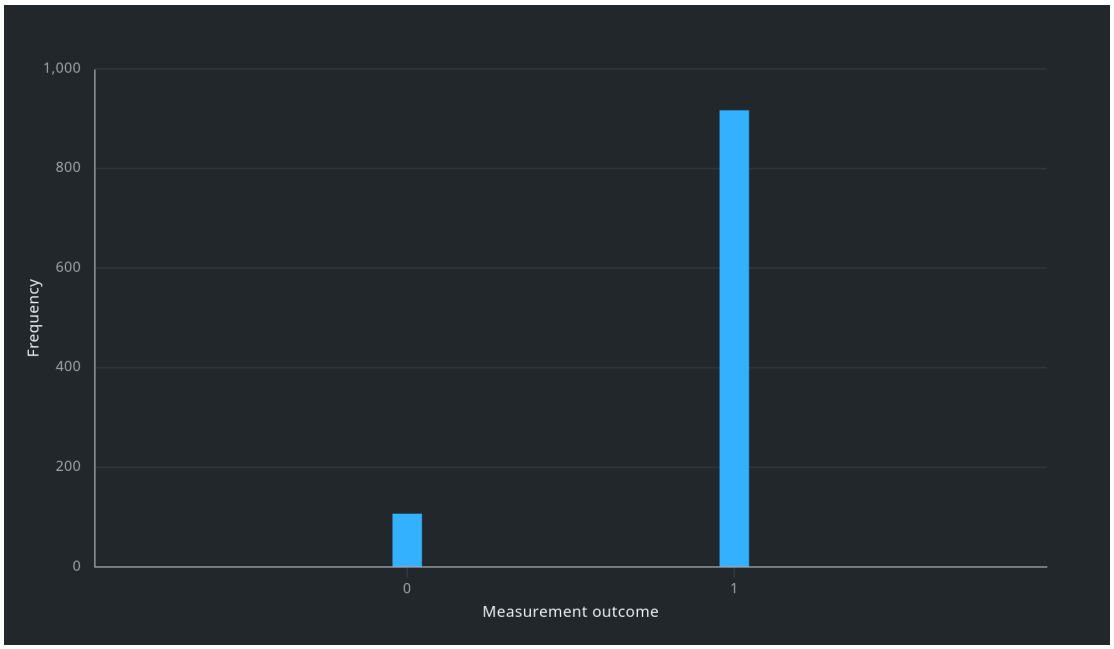
Run 1:



Run 2:



Run 3:

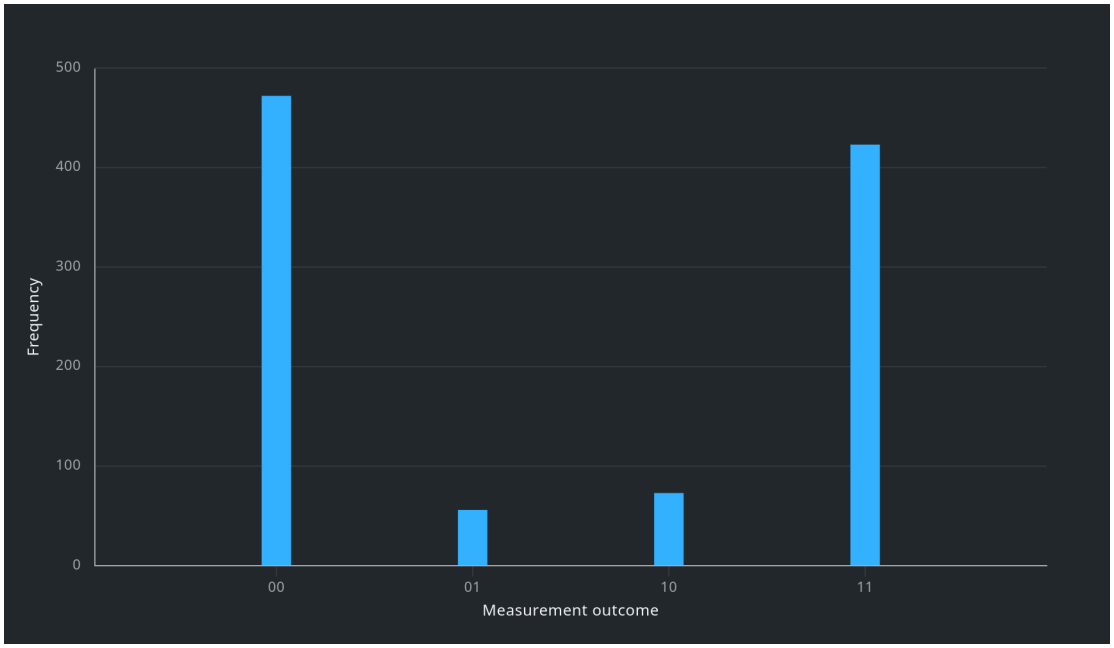


Run	|0⟩ Counts	|1⟩ Counts	Job ID
Run 1	113	911	d651cqns6ggc73fjoqv0
Run 2	119	905	d651gcbtraac73bj3iug
Run 3	107	917	d651lmrtraac73bj3psg
Average	113	911	NA

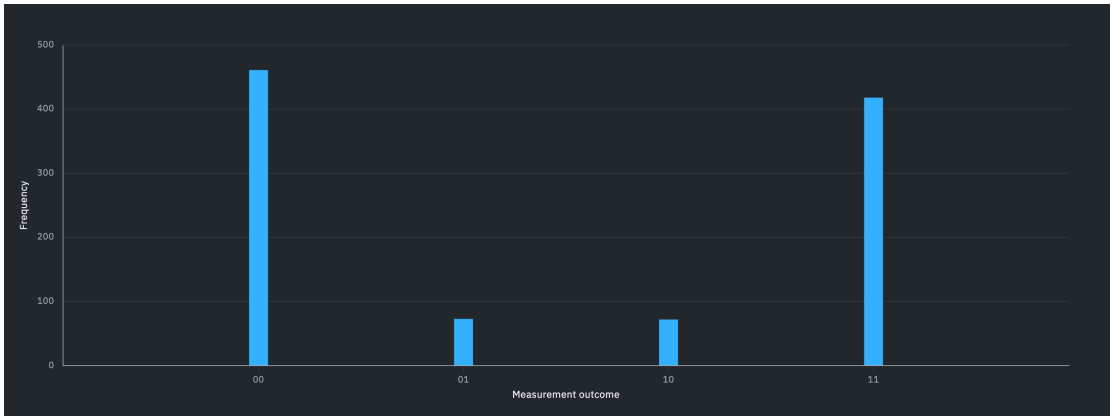
Circuit 3: 2-Qubit Bell State

Probability Histograms

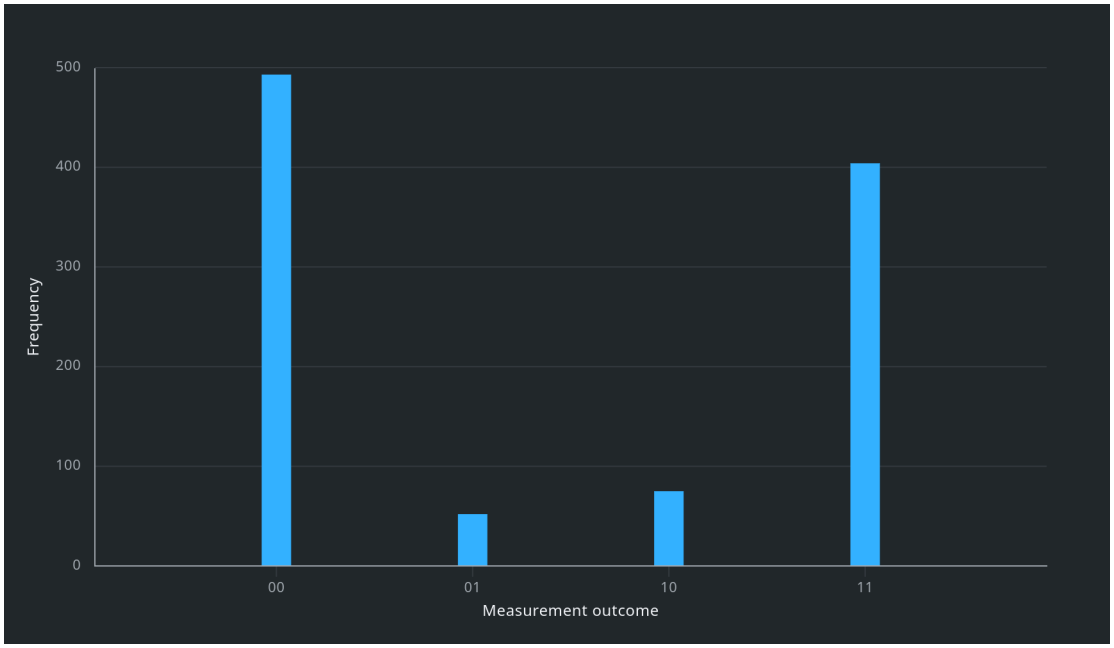
Run 1:



Run 2:



Run 3:



Run	|00⟩ Counts	|01⟩ Counts	|10⟩ Counts	|11⟩ Counts	Job ID

Run 1	472	56	73	423	d651jfio8gvs73f55hhg
Run 2	461	73	72	418	d651jbbc4tus73fggbcg
Run 3	493	52	75	404	d651jvio8gvs73f55ifg
Average	475.33	60.33	73.33	415	NA

In [28]:
```
import numpy as np

shots = 1024

## Simulate Circuit 1 H-gate ##

Hgate_outcomes = np.random.choice(["|0>","|1>"], shots, p = [.5,.5])
kets, counts = np.unique(Hgate_outcomes, return_counts = True)
print("==== Circuit 1 H-Gate ====")
print(f"{kets[0]} : {counts[0]/shots : .2%}, {kets[1]} : {counts[1]/shots

# Xgate_outcomes = np.random.choice(["|0>","|1>"], shots, p = [0,1])
# kets, counts = np.unique(Xgate_outcomes, return_counts = True)
print("==== Circuit 2 X-Gate ====")
print(f"{kets[0]} : {0/shots : .2%}, {kets[1]} : {shots/shots : .2%}")


Bell_outcomes = np.random.choice(["|00>","|11>"], shots, p = [.5,.5])
kets, counts = np.unique(Bell_outcomes, return_counts = True)
print("==== Circuit 3 Bell ====")
print(f"{kets[0]} : {counts[0]/shots : .2%}, {kets[1]} : {counts[1]/shots
```

```
==== Circuit 1 H-Gate ====
|0> :  49.22%, |1> :  50.78%
==== Circuit 2 X-Gate ====
|0> :  0.00%, |1> :  100.00%
==== Circuit 3 Bell ====
|00> :  52.34%, |11> :  47.66%
```

Key Question: Why only |00⟩ + |11⟩?

The two states are fully entagled, leading to only two results, if the first bit is measured as 1 the second one has to be 1 as well. Theoretically this works, but in the actual run there are errors that prevent this from working consistantly.

Discussion Questions

Why do histograms vary between runs? The runs are for 1024 shots. Each run is independent on other runs, to ensure a 50/50 split we would have many more shots.

Circuit 3: Why no |01⟩ or |10⟩ states? The expected output for Circuit 3 states that the only 2 outputs are |00⟩ or |11⟩. This is due to entanglement as talked about above.

How many shots improve statistics?This depends on the starting number of shots. Here we have 1024, which gets us pretty close to average. The closer you get to the expected outcome the more shots you have to add (exponential growth). Below I messed with the number of shots and observed that running it with 4096 shots lead to noticibly more stable results.

In [53]:
```
Hgate_outcomes = np.random.choice(["|0>","|1>"], shots, p = [.5,.5])
kets, counts = np.unique(Hgate_outcomes, return_counts = True)
print("==== Circuit 1 H-Gate ====")
print(f"{kets[0]} : {counts[0]/shots : .2%}, {kets[1]} : {counts[1]/shots
```

```
==== Circuit 1 H-Gate ====
|0> :  48.73%, |1> :  51.27%
```

In [44]:
```
shots2 =1024*4
Hgate_outcomes = np.random.choice(["|0>","|1>"], shots2, p = [.5,.5])
kets, counts = np.unique(Hgate_outcomes, return_counts = True)
print("==== Circuit 1 H-Gate ====")
print(f"{kets[0]} : {counts[0]/shots2 : .2%}, {kets[1]} : {counts[1]/shot
```

```
==== Circuit 1 H-Gate ====
|0> :  50.02%, |1> :  49.98%
```